

BNP-WALK: A Pedagogical Manual for Inferring Kinesin and Other Molecular-Motor Steps from 2D Tracked Trajectories

Documentation and mathematical guide to the supplied Julia MCMC implementation

Based on the supplied Julia inference code and `plot_mcmc_output.jl`

August 25, 2026

Contents

1	Installation and first-time setup	4
1.1	Install Julia	4
1.2	Create a clean analysis environment	4
1.3	Quick package test	4
1.4	Run the inference code	5
1.5	Critical setup: specify a realistic typical dwell time	5
1.6	Run the posterior plotting code	6
1.7	Optional: reproduce the environment later	6
2	Optional preprocessing with <code>process_data.jl</code>	6
3	Purpose and scope	7
4	The four source files and program architecture	7
5	Input data and coordinate preprocessing	8
5.1	Relation to quadrant-detector tracking	9
6	Latent representation of a stepping trajectory	9
7	Physical model: the two-dimensional Ornstein–Uhlenbeck process	10
7.1	Stiffness matrix	10
7.2	Continuous-time dynamics	10
7.3	Exact OU covariance used in the code	10
7.4	How this appears in the Julia code	11
8	Bayesian structure	11
9	Model modes: parametric versus nonparametric	11
10	Priors and their physical interpretation	12
10.1	Dwell time prior	12
10.2	Jump-size prior	12
10.3	Stiffness prior	12
10.4	Diffusion coefficient prior	13

11 Initialization	13
11.1 Initial number and placement of dwells	13
11.2 Initial change-point times	13
12 The MCMC update cycle	14
12.1 Metropolis–Hastings logic	14
12.2 Simulated annealing	15
13 Update 1: sampling dwell positions	15
13.1 Proposal	15
13.2 Likelihood contribution	15
13.3 Position priors	15
13.4 Accepted versus rejected position proposal	16
14 Update 2: sampling stiffness	16
14.1 Proposal for diagonal stiffnesses	16
14.2 Proposal for the cross term	16
14.3 What is recomputed after a stiffness proposal?	16
15 Update 3: sampling the diffusion coefficient	17
16 Update 4: dwell-time hyperparameters	17
17 Update 5: change-point times	18
17.1 Why use dwell durations?	18
17.2 Proposal validity rule	18
17.3 Asymmetric proposal correction	18
18 Update 6: active/inactive change points	18
18.1 Prior probability of activation	19
18.2 Turning a change point on	19
18.3 Turning a change point off	19
18.4 Why this is step detection	19
19 Optional trajectory collapse rule	20
20 Data validity and numerical safeguards	20
20.1 Positive-definite stiffness	20
20.2 Positive-definite OU covariance	20
20.3 Position range restrictions	20
20.4 Dwell-duration restrictions	21
20.5 Maximum jump restriction	21
21 MCMC bookkeeping and acceptance diagnostics	21
22 Sampling schedule and saved posterior state	21
23 Output file structure	22
24 Plotting and what the figures mean	22
24.1 Panel 1: trajectory and inferred dwell positions	22
24.2 Panel 2: change-point locations in time	23
24.3 Panel 3: log-posterior trace	23

25	Posterior plotting workflow: <code>plot_mcmc_output.jl</code>	23
25.1	Packages used by the plotting script	23
25.2	File locations	23
25.3	Reading the posterior HDF5 file	24
25.4	Reconstructing active dwells	24
25.5	The Gamma fit shown in the dwell-time panel	24
25.6	Panel 1: posterior spatial density	25
25.7	Panel 2: change-point histogram	25
25.8	Panel 3: dwell-time distribution	25
25.9	Panel 4: jump-size distribution	25
25.10	Panels 5 and 6: dwell-time hyperparameters	25
25.11	How to run the plotting script safely	26
25.12	Current plotting-script limitations	26
26	Ground-truth mode	26
27	A concrete conceptual example	27
28	Practical interpretation of the inferred quantities	27
28.1	Step positions	27
28.2	Step sizes	27
28.3	Step times and dwell times	27
28.4	Stiffness	28
28.5	Diffusion	28
29	Important implementation details and caveats	28
29.1	The detector model is absent	28
29.2	Coordinate system matters	28
29.3	The prior scale is experiment-specific	28
29.4	The first and last 100 samples are special	28
29.5	The MCMC state is repeatedly reinitialized after saving	28
29.6	Acceptance-ratio code contains asymmetric behavior between active and inactive states	29
29.7	There are legacy/commented likelihood implementations	29
30	How the complete algorithm can be summarized mathematically	29
31	Recommended workflow for a real experimental trajectory	29
32	A compact pseudocode version	30
33	One-page conceptual map	31
34	Source-code map for readers learning the program	31
35	Final perspective	32
A	Current parameter set in the updated code	32
B	Suggested downstream posterior analysis	34

1 Installation and first-time setup

This section is intended for a new user who has never run the analysis before. The supplied inference code uses Julia's standard-library modules together with a small set of registered packages. The current stable Julia release should be installed from the official Julia distribution; the Julia project recommends `juliaup` for typical installations and also provides official binaries for manual installation.^[1, 2] `Pkg`, Julia's built-in package manager, is used to install registered packages, and multiple packages can be added in one command.^[3]

1.1 Install Julia

The recommended route is to install Julia using `juliaup`. Visit the official Julia downloads page and follow the installer for your operating system. At the time this manual was regenerated, the Julia project listed Julia 1.12.6 as the current stable release.^[1]

After installation, open a terminal and verify that Julia is available:

```
julia --version
```

You should see a Julia version beginning with 1.. The exact current minor/patch version may change over time; the code in this manual does not depend on a single patch release.

1.2 Create a clean analysis environment

A reproducible Julia workflow is easiest to maintain in a dedicated project environment. In the directory containing the four inference files and the plotting script, start Julia and create/activate an environment:

```
julia
```

Then press `]` to enter Julia's package manager and run:

```
activate .  
add HDF5 Distributions SpecialFunctions CairoMakie KernelDensity Plots  
status
```

`Pkg` records dependencies in `Project.toml` and `Manifest.toml`. These files are useful when the analysis is moved to another machine because the environment can subsequently be recreated with `instantiate`.^[3, 4]

The inference program loads

```
using Random, Distributions, SpecialFunctions, Statistics, LinearAlgebra  
using HDF5
```

Of these, `Random`, `Statistics`, and `LinearAlgebra` are Julia standard-library modules; they do not need to be added as third-party packages. The externally installed packages needed by the inference code are `HDF5`, `Distributions`, and `SpecialFunctions`. The supplied posterior plotting script additionally requires `CairoMakie` and `KernelDensity`. The optional preprocessing script `process_data.jl` uses `HDF5` and `Plots`. Thus, installing the combined set above is sufficient for preprocessing, inference, and posterior visualization.

1.3 Quick package test

From the Julia REPL, verify that the environment can load everything:

```
using Random, Distributions, SpecialFunctions, Statistics, LinearAlgebra  
using HDF5  
using CairoMakie, KernelDensity, Plots  
println("All required packages loaded successfully.")
```

If this command succeeds without an error, the Julia side of the installation is complete.

1.4 Run the inference code

Place the following files together:

```
input_parameters.jl
function_layer_1.jl
function_layer_2.jl
main.jl
```

The HDF5 input file identified by `file_id` should also be accessible from the working directory chosen by `working_directory`. The simplest way to launch the complete inference is directly from a terminal. Change into the directory containing the code and type:

```
julia main.jl
```

This is the recommended command for routine use. Julia executes `main.jl`, which loads `input_parameters.jl`, `function_layer_1.jl`, and `function_layer_2.jl`; the program then calls `sampler()` at the end of `main.jl`, so the complete MCMC analysis starts automatically. An interactive alternative is to start the Julia REPL and type `include("main.jl")`.

1.5 Critical setup: specify a realistic typical dwell time

Before launching the nonparametric analysis, the user should provide a physically reasonable approximate average dwell time through `typical_dwell_time` in `input_parameters.jl`. This is a **critical setup parameter**, not merely a cosmetic initial guess.

In the current nonparametric implementation, `main.jl` automatically determines the maximum number of candidate dwells from the duration of the analyzed trajectory and the specified typical dwell time:

$$J_{\max} = 5 \left\lceil \frac{N \Delta t}{\tau_{\text{typ}}} \right\rceil, \quad (1)$$

where, in the code, N is represented by `n_time_points`, $\Delta t = dt$, and $\tau_{\text{typ}} = \text{typical_dwell_time}$. The implementation is

```
const max_n_dwells::Int64 =
    5 * ceil(n_time_points * dt / typical_dwell_time)
```

The factor of five gives the nonparametric sampler room to consider substantially more dwells than would be predicted by the approximate average dwell time alone.

Consequently, an unrealistically *small* `typical_dwell_time` creates an unnecessarily large `max_n_dwells` and therefore a much larger candidate change-point space, with increased memory and computational cost. An unrealistically *large* value can make `max_n_dwells` too restrictive and can prevent the model from representing a trajectory containing many short dwells. The same quantity also initializes the Gamma dwell-time mean and sets the scale of its hyperprior. Users should therefore estimate the approximate average dwell time from prior experimental knowledge, a preliminary inspection of the trace, or a comparable dataset before beginning a production run. It need not be exact, but it should be of the correct physical order of magnitude.

For the supplied parameter file,

```
const typical_dwell_time = 3500*10*dt
```

which equals 0.0175 s for $dt = 5 \times 10^{-7}$ s.

1.6 Run the posterior plotting code

After the inference has produced `mcmc_output_<file_id>.h5`, install the plotting packages listed above and run:

```
include("plot_mcmc_output.jl")
```

Before running it, edit `current_directory` in the plotting script so that it points to the directory containing the input HDF5 file and the MCMC output HDF5 file. The supplied script currently contains an absolute path specific to the original analysis machine and therefore must be changed on a different computer.

1.7 Optional: reproduce the environment later

If `Project.toml` and `Manifest.toml` are already present, a later user can activate the directory and instantiate the recorded environment:

```
julia
]\nactivate .
instantiate
```

This is preferable to repeatedly installing packages by hand when reproducing an analysis.

2 Optional preprocessing with `process_data.jl`

The supplied `process_data.jl` is a small experiment-specific preprocessing utility that converts the laboratory trajectory HDF5 layout into the simpler HDF5 format expected by the inference code. It is useful when starting from the original tracking-analysis file; it is not required if a suitable `<file_id>.h5` containing `x(m)` and `y(m)` already exists.

The script opens a source trajectory file, reads `analysis/idx_valid`, `analysis/xy`, the sampling interval `analysis/t/xy/dt`, and the initial time `analysis/t/xy/t_0`. It then selects a user-defined time window using `good_data_start_time_index` and `good_data_end_time_index`, extracts the first tracked object's two Cartesian coordinates, and writes a new file named `processed_trajectory_invivo.h5`. The output contains

```
x(m)
y(m)
dt(s)
```

which supplies the `x(m)` and `y(m)` datasets consumed by `get_data()` in the inference workflow. Note that the inference currently takes `dt` from `input_parameters.jl`; the stored `dt(s)` is useful metadata but is not automatically substituted for that parameter.

Before using this preprocessing script on a new dataset, edit its absolute `working_directory`, source filename, desired good-data time interval, and output filename as needed. It can then be run from a terminal with

```
julia process_data.jl
```

The script also saves three quick diagnostic plots—`xy_plot.png`, `tx_plot.png`, and `ty_plot.png`—using `Plots.jl`. These are intended for checking the selected trajectory segment before committing to the much more expensive MCMC analysis. The current script contains experiment-specific choices (including hard-coded paths, time-window choices, and a temporary `data_range = 1:7000` used for plotting), so users should inspect and edit it rather than treating it as a universal importer.

A practical end-to-end sequence is therefore:

1. optionally run `julia process_data.jl` to create and visually inspect the processed trajectory;
2. edit `input_parameters.jl`, paying particular attention to `dt`, `data_range`, and `typical_dwell_time`;
3. run the inference with `julia main.jl`;
4. after posterior samples have been saved, run `plot_mcmc_output.jl` for posterior visualization.

3 Purpose and scope

This document explains, step by step, the supplied Julia program for analyzing a two-dimensional trajectory of a labeled molecular motor, such as kinesin, moving along a microtubule while its position is measured from a gold nanoparticle and quadrant-detector tracking. The program receives an already tracked trajectory $x(t), y(t)$ and performs Bayesian inference on a piecewise dynamical model.

A crucial distinction is important at the outset: the supplied code does *not* simulate the quadrant detector, the optical point-spread function, photon statistics, or the conversion from quadrant-detector voltages to Cartesian position. Instead, it assumes that this upstream tracking pipeline has already produced the position time series and that those positions are available in an HDF5 file. The first function in the analysis reads the datasets `x(m)` and `y(m)`, then shifts both coordinates so that the first point is the origin. Thus, everything documented here begins with the measured trajectory and focuses on statistical inference from that trajectory.

The main inferential question is:

Given a noisy 2D trajectory, where are the motor’s dwell states, when do transitions occur, where are the latent dwell positions, and what mechanical/dynamical parameters best explain the observed fluctuations within each dwell?

The implementation answers this question with an MCMC algorithm that repeatedly proposes and accepts or rejects changes to:

1. the latent dwell positions (x_j, y_j) ,
2. the stiffness matrix of each dwell,
3. the diffusion coefficient D ,
4. the parameters controlling the prior distribution of dwell durations,
5. the change-point times, and
6. in the nonparametric mode, which candidate change points are actually active.

The key computational model is a two-dimensional Ornstein–Uhlenbeck (OU) process with a full 2×2 stiffness matrix. The use of a full matrix is important because the code permits correlated x - and y -motion through the off-diagonal stiffness term k_{xy} .

4 The four source files and program architecture

The supplied implementation is divided into four logical layers.

File	Role
input_parameters.jl	Global constants controlling the data set, physical constants, priors, proposal widths, annealing schedule, collapse rules, and number of MCMC samples.
main.jl	Top-level program. Loads packages and files, loads the data, computes data summaries, allocates MCMC state arrays, performs the MCMC loop, reports diagnostics, plots, saves samples, and reinitializes after each saved block.
function_layer_1.jl	High-level inference functions: data loading, initialization, MCMC updates for loads/change-point activation, dwell-time hyperparameters, change-point times, positions, stiffness, diffusion, plotting, and HDF5 output.
function_layer_2.jl	Mathematical helper functions: active/inactive change-point bookkeeping, distribution log-densities, OU matrix preparation, and the core 2D OU log-likelihood.

Table 1: Logical structure of the supplied implementation.

The top-level control flow is approximately

```

load packages
include input_parameters
initialize RNG
include function_layer_1 and function_layer_2
optionally load ground truth
read and center x,y trajectory
compute derived data summaries
compute prior probability that a candidate change point is active
enter sampler()
    allocate latent variables and storage
    initialize model
    build OU matrices
    for each MCMC draw
        compute annealing temperature
        sample positions
        sample stiffness
        sample diffusion coefficient
        sample dwell-time hyperparameters
        sample change-point times
        optionally sample which change points are active
        optionally collapse very small jumps
        update acceptance diagnostics
        store posterior/log-posterior information
        periodically plot and save HDF5 output
        periodically reinitialize for the next saved block
return

```

5 Input data and coordinate preprocessing

The function `get_data()` constructs the HDF5 filename from `working_directory`, `file_id`, and the `.h5` extension. It reads $x(m)$ and $y(m)$ and then performs

$$x_i \leftarrow x_i - x_1, \quad y_i \leftarrow y_i - y_1. \quad (2)$$

Thus the inferred trajectory is expressed relative to the first detected position. This is not an inference step; it is a coordinate convention.

The parameter file currently sets

$$\Delta t = 5 \times 10^{-7} \text{ s}, \quad (3)$$

so the sampling interval is $0.5 \mu\text{s}$. The updated parameter file selects `data_range = 1:920000`; the source comment notes that `Colon()` can be used to select the full data set.

The main file computes the number of observations, coordinate extrema, and the mean position during the first and last 100 frames. Those means are later used as weak endpoint position priors when sampling the latent dwell positions.

5.1 Relation to quadrant-detector tracking

The physical experiment can be pictured as

motor + microtubule $\longrightarrow$ gold nanoparticle label $\longrightarrow$ optical detection / quadrant detector $\longrightarrow (x_i, y_i) \longrightarrow$ th

The supplied Julia program starts at the final (x_i, y_i) output of that chain. Consequently, measurement noise caused by the detector is represented only indirectly through the OU residual covariance/model mismatch; there is no explicit detector-noise calibration parameter or forward observation model in the supplied code.

6 Latent representation of a stepping trajectory

The central latent representation is a sequence of dwell states separated by change points. Let

$$\tau_1 < \tau_2 < \cdots < \tau_{J+1} \quad (4)$$

be change-point times. The interval

$$[\tau_j, \tau_{j+1}] \quad (5)$$

is treated as dwell j , associated with a latent reference position

$$\mathbf{r}_j = \begin{pmatrix} x_j \\ y_j \end{pmatrix}. \quad (6)$$

The displacement between consecutive latent dwells is

$$\delta_j = \mathbf{r}_{j+1} - \mathbf{r}_j, \quad \ell_j = \|\delta_j\|, \quad (7)$$

where ℓ_j is the inferred jump size.

The arrays called `loads` represent whether each candidate change point is active. A nonzero entry means that the candidate state is active; a zero entry means inactive. The first and last change points are treated as always active. The helper function `get_active_inactive_loads!()` rebuilds two index lists:

$$\text{loads_active} = \{j : \text{loads}_j \neq 0\}, \quad (8)$$

and

$$\text{loads_inactive} = \{j : \text{loads}_j = 0\}. \quad (9)$$

This bookkeeping is what permits the nominal grid of a data-dependent maximum number dwells to represent fewer actual dwells in the nonparametric model.

7 Physical model: the two-dimensional Ornstein–Uhlenbeck process

7.1 Stiffness matrix

For each dwell, the code defines

$$\mathbf{K}_j = \begin{pmatrix} k_{xx,j} & k_{xy,j} \\ k_{xy,j} & k_{yy,j} \end{pmatrix}. \quad (10)$$

The diagonal elements describe restoring stiffness along the two coordinate axes. The off-diagonal element couples the two coordinates.

Positive definiteness is required. The determinant is

$$\det(\mathbf{K}_j) = k_{xx,j}k_{yy,j} - k_{xy,j}^2. \quad (11)$$

The likelihood function rejects a dwell whenever this determinant is non-finite or non-positive.

7.2 Continuous-time dynamics

The OU process used by the code corresponds to linear restoring dynamics with diffusion. In matrix notation, the relaxation matrix is proportional to

$$\frac{D}{k_B T} \mathbf{K}_j. \quad (12)$$

For a single time step Δt , the code constructs the exact mean propagator

$$\mathbf{A}_j = \exp \left[-\frac{D\Delta t}{k_B T} \mathbf{K}_j \right]. \quad (13)$$

This is an exact matrix exponential, rather than an Euler approximation.

If $\mathbf{z}_i = \mathbf{r}_i^{\text{det}} - \mathbf{r}_j$ denotes the observed displacement from the inferred dwell position, the conditional mean is represented as

$$\mathbb{E}[\mathbf{z}_i \mid \mathbf{z}_{i-1}] = \mathbf{A}_j \mathbf{z}_{i-1}. \quad (14)$$

Therefore the innovation or residual is

$$\mathbf{r}_i = \mathbf{z}_i - \mathbf{A}_j \mathbf{z}_{i-1}. \quad (15)$$

7.3 Exact OU covariance used in the code

The code computes

$$\boldsymbol{\Sigma}_j = k_B T \mathbf{K}_j^{-1} (\mathbf{I} - \mathbf{A}_j \mathbf{A}_j). \quad (16)$$

Because the stiffness matrix is symmetric and $\mathbf{A}_j$ is a matrix exponential of $\mathbf{K}_j$, these matrices commute in the present model. The implementation then stores $\boldsymbol{\Sigma}_j^{-1}$ and $\det(\boldsymbol{\Sigma}_j)$ for repeated likelihood evaluations.

The likelihood is therefore the sum of bivariate Gaussian log densities,

$$\log L_j = \sum_{i \in \mathcal{T}_j} \left[-\frac{1}{2} \log((2\pi)^2 \det \boldsymbol{\Sigma}_j) - \frac{1}{2} \mathbf{r}_i^T \boldsymbol{\Sigma}_j^{-1} \mathbf{r}_i \right], \quad (17)$$

where $\mathcal{T}_j$ is the set of frames assigned to the transition from the previous detected position to each current frame within dwell j .

7.4 How this appears in the Julia code

The function `prepare_OU!()` computes the stiffness determinant and inverse, the matrix exponential, the OU covariance, and the covariance inverse. The likelihood function `get_log_likelihood!()` then loops over the relevant frames and evaluates the residual and quadratic form.

A simplified rendering of its core calculation is

```
residual = observed_position - reference_position
           - dynamics_matrix * previous_trap_extension

quad_form = residual' * inv_OU_covariance * residual

log_likelihood +=
    -0.5 * log((2*pi)^2 * det_OU_covariance)
    -0.5 * quad_form
```

The source code evaluates the two components explicitly rather than forming a temporary 2-vector.

8 Bayesian structure

The sampler is trying to explore a posterior distribution of the generic form

$$\mathcal{P}(\Theta \mid \mathbf{y}) \propto L(\mathbf{y} \mid \Theta) p(\Theta), \quad (18)$$

where Θ collectively contains the latent dwell structure, dwell positions, stiffness parameters, diffusion coefficient, dwell-time hyperparameters, and related state variables.

Since the implementation is MCMC, it does not return only one optimized parameter set. Instead it generates a sequence of samples from the posterior (subject to burn-in, annealing, initialization, and storage choices). Posterior summaries should therefore be computed from the saved sample arrays rather than from a single iteration.

9 Model modes: parametric versus nonparametric

The parameter `modeling_choice` is currently set to

```
const modeling_choice = "nonparametric"
```

In the nonparametric mode, the code permits a data-dependent maximum number candidate dwells and uses a binary active/inactive representation for internal change points. The expected number of dwells is set to 10. The current code computes the maximum number of candidate dwells dynamically as

$$J_{\max} = 5 \left\lceil \frac{n \Delta t}{\mu_{\text{dwell}}} \right\rceil, \quad (19)$$

where the total duration is inferred from the number of data points. With the current values $n = 920000$, $\Delta t = 5 \times 10^{-7}$ s, and $\mu_{\text{dwell}} = 0.0175$ s, this gives $J_{\max} = 135$ candidate dwells and therefore 136 candidate change points.

In the parametric mode, the number of dwells is fixed at the requested value and `dwell_collapse_required` is forced to false.

The nonparametric interpretation is best understood as follows. Start with a generous maximum number of candidate change points, then allow the MCMC to deactivate candidates that are not supported by the data. Hence the posterior can contain, for example, 8 active dwells even though the latent arrays were allocated for a much larger data-dependent maximum.

10 Priors and their physical interpretation

10.1 Dwell time prior

The duration of each active dwell is assigned a Gamma density with shape α and scale chosen so that the mean is μ :

$$\tau_j - \tau_{j-1} \sim \text{Gamma}\left(\alpha, \frac{\mu}{\alpha}\right). \quad (20)$$

Thus

$$\mathbb{E}[\Delta\tau] = \mu. \quad (21)$$

The supplied initial values are

$$\mu = 3500 \times 10 \times \Delta t, \quad (22)$$

$$\alpha = 2. \quad (23)$$

Numerically, with $\Delta t = 5 \times 10^{-7}$ s, the typical dwell time is 0.0175 s.

The hyperparameters α and μ themselves are sampled. Their proposals are made on the logarithmic scale to preserve positivity.

10.2 Jump-size prior

The position sampler uses a Gaussian prior on the magnitude of jumps:

$$\ell_j \sim \mathcal{N}(\ell_0, \sigma_\ell^2), \quad (24)$$

with the supplied values

$$\ell_0 = 8 \text{ nm}, \quad \sigma_\ell = 8 \text{ nm}. \quad (25)$$

This prior favors step-like changes of order a few nanometers but is intentionally broad.

10.3 Stiffness prior

For active dwells, the diagonal stiffness elements have Gamma priors,

$$k_{xx}, k_{yy} \sim \text{Gamma}\left(a_k, \frac{k_0}{a_k}\right), \quad (26)$$

with

$$k_0 = 7.5 \times 10^{-5} \text{ N/m}, \quad a_k = 1. \quad (27)$$

The correlation-like quantity

$$\rho_{xy} = \frac{k_{xy}}{\sqrt{k_{xx}k_{yy}}} \quad (28)$$

has a Uniform($-1, 1$) prior. The code reconstructs

$$k_{xy} = \rho_{xy} \sqrt{k_{xx}k_{yy}}, \quad (29)$$

which automatically enforces

$$|k_{xy}| < \sqrt{k_{xx}k_{yy}} \quad (30)$$

when $|\rho_{xy}| < 1$, thereby maintaining positive definiteness of the stiffness matrix.

10.4 Diffusion coefficient prior

The diffusion coefficient has a Gaussian prior

$$D \sim \mathcal{N}(D_0, \sigma_D^2), \quad (31)$$

with

$$D_0 = 10^{-13} \text{ m}^2/\text{s}, \quad \sigma_D = 100D_0. \quad (32)$$

The proposal is made in $\log D$, guaranteeing a positive proposed value.

11 Initialization

The function `initialize_parameters!()` creates the starting state for the MCMC. The initialization is intentionally simple and deterministic enough to provide a plausible starting trajectory.

11.1 Initial number and placement of dwells

For the nonparametric model, the first and last candidate change points are active and intermediate candidates are populated so that the initial representation spans the allowed set of possible states.

The observed trajectory's net displacement is

$$L_{\text{net}} = \sqrt{(x_N - x_1)^2 + (y_N - y_1)^2}. \quad (33)$$

An initial average jump magnitude is then chosen as

$$\ell_{\text{init}} = \frac{L_{\text{net}}}{N_{\text{dwell}, \text{max}}}. \quad (34)$$

Candidate positions are initially placed along the straight line joining the first and last observed points. If

$$\theta = \text{atan2}(y_N - y_1, x_N - x_1), \quad (35)$$

then the initial positions are

$$x_j = x_1 + (j - 1)\ell_{\text{init}} \cos \theta, \quad (36)$$

$$y_j = y_1 + (j - 1)\ell_{\text{init}} \sin \theta. \quad (37)$$

This is merely the starting point for the MCMC, not the final inferred trajectory.

11.2 Initial change-point times

The available time is divided approximately uniformly across the initial set of candidate dwells. If the data have N frames, the total elapsed time is

$$T = (N - 1)\Delta t. \quad (38)$$

The initial candidate dwell time is approximately

$$\Delta\tau_{\text{init}} = \frac{T}{N_{\text{dwell}, \text{max}}}. \quad (39)$$

The code stores both continuous times and integer frame indices. For a proposed time τ_j , the corresponding frame index is approximately

$$i_j = \left\lceil \frac{\tau_j}{\Delta t} \right\rceil + 1. \quad (40)$$

This distinction between continuous change-point time and the discrete observation index is central to the implementation.

12 The MCMC update cycle

Each MCMC draw updates the state in the following sequence:

1. positions,
2. stiffness parameters,
3. diffusion coefficient,
4. dwell-time hyperparameters,
5. change-point times,
6. active/inactive change-point indicators (during the designated nonparametric period),
7. optional collapse of very small adjacent jumps.

The order matters computationally because each update modifies quantities that affect the next update's likelihood calculations.

12.1 Metropolis–Hastings logic

Most parameter updates follow a common pattern. Let Θ denote the current state and Θ' a proposed state. The code computes

$$\log \mathcal{P}_{\text{old}} = \log L(\Theta \mid y) + \log p(\Theta), \quad (41)$$

and

$$\log \mathcal{P}_{\text{new}} = \log L(\Theta' \mid y) + \log p(\Theta'). \quad (42)$$

For symmetric proposals, the log acceptance ratio is essentially

$$\log H = \log \mathcal{P}_{\text{new}} - \log \mathcal{P}_{\text{old}}. \quad (43)$$

The proposal is accepted if

$$\log H \geq \log u, \quad u \sim \text{Uniform}(0, 1). \quad (44)$$

For asymmetric proposals, the code additionally includes the logarithm of the backward proposal probability minus the forward proposal probability.

12.2 Simulated annealing

The code uses a temperature parameter

$$T_A(d) = 1 + (T_0 - 1) \exp \left[-\frac{d}{\lambda_A} \right], \quad (45)$$

where the implementation restarts the exponential schedule within each save-burn-in block. The current values are

$$T_0 = 20000, \quad \lambda_A = 1000. \quad (46)$$

The posterior difference is scaled by $1/T_A$. At high temperature, unfavorable moves are easier to accept, helping exploration of difficult, multimodal posterior landscapes. As the temperature approaches one, the sampler becomes an ordinary Metropolis–Hastings procedure.

13 Update 1: sampling dwell positions

The function `sample_positions!()` updates the latent reference position of each dwell.

13.1 Proposal

For a current position (x_j, y_j) , the proposal is

$$x'_j \sim \mathcal{N}(x_j, s_x^2), \quad (47)$$

$$y'_j \sim \mathcal{N}(y_j, s_x^2), \quad (48)$$

with

$$s_x = 0.5 \text{ nm}. \quad (49)$$

The proposal is symmetric.

13.2 Likelihood contribution

The proposed location is evaluated with the OU likelihood for the frames assigned to that dwell. The code also rejects a proposed reference point if it falls outside the observed coordinate range within the relevant time interval:

$$\min(x_{\text{obs}}) < x'_j < \max(x_{\text{obs}}), \quad (50)$$

and similarly for y . This prevents latent dwell positions from wandering into obviously unsupported regions.

13.3 Position priors

The position sampler uses several complementary priors:

1. neighboring jump magnitudes are encouraged to remain near the typical jump size;
2. each dwell location is encouraged to remain near the observed mean position during its own time window;
3. the first and last dwell receive additional anchoring to the mean of the first/last 100 observations.

For an interior dwell, the jump-size prior is schematically

$$\log p(\mathbf{r}_j) \supset \log \mathcal{N}(\ell_{j-1}; \ell_0, \sigma_\ell^2) + \log \mathcal{N}(\ell_j; \ell_0, \sigma_\ell^2). \quad (51)$$

The code also uses

$$x_j \sim \mathcal{N}(\bar{x}_j, \sigma_\ell^2/4), \quad y_j \sim \mathcal{N}(\bar{y}_j, \sigma_\ell^2/4), \quad (52)$$

where $(\bar{x}_j, \bar{y}_j)$ is the mean observed position in that dwell's frame window.

13.4 Accepted versus rejected position proposal

If the log-posterior improves sufficiently according to the Metropolis criterion, the new position is retained. Otherwise, the old position is restored.

14 Update 2: sampling stiffness

The function `sample_stiffness!()` updates k_{xx} , k_{yy} , and k_{xy} for every active dwell, and also proposes values for inactive candidate dwells.

14.1 Proposal for diagonal stiffnesses

The proposed diagonal parameters are sampled from Gamma proposal distributions centered on their current values in the Gamma-family parameterization used by the code:

$$k'_{xx} \sim \text{Gamma}\left(a_q, \frac{k_{xx}}{a_q}\right), \quad k'_{yy} \sim \text{Gamma}\left(a_q, \frac{k_{yy}}{a_q}\right), \quad (53)$$

where $a_q = 500$. A larger value makes the proposal more concentrated around the current value.

14.2 Proposal for the cross term

Rather than proposing k_{xy} directly, the code works with

$$\rho_{xy} = \frac{k_{xy}}{\sqrt{k_{xx}k_{yy}}}. \quad (54)$$

It maps this variable to

$$u = \frac{\rho_{xy} + 1}{2} \in (0, 1) \quad (55)$$

and uses a Beta proposal for u . It then reconstructs

$$k'_{xy} = \rho'_{xy} \sqrt{k'_{xx}k'_{yy}}. \quad (56)$$

Because the proposal is state-dependent, the Metropolis–Hastings ratio contains both forward and backward proposal terms.

14.3 What is recomputed after a stiffness proposal?

The code immediately rebuilds the OU matrices for the affected dwell using `prepare_OU!()`. This is necessary because changing stiffness changes:

$$\mathbf{K} \rightarrow \mathbf{K}^{-1} \rightarrow \mathbf{A} \rightarrow \boldsymbol{\Sigma} \rightarrow \log L. \quad (57)$$

If the proposal is rejected, the old stiffness values are restored and the OU matrices are rebuilt again.

15 Update 3: sampling the diffusion coefficient

The function `sample_diffusion_coeff()` proposes the diffusion coefficient in log space:

$$\log D' \sim \mathcal{N}(\log D, s_D^2), \quad (58)$$

with supplied proposal width

$$s_D = 10^{-2}. \quad (59)$$

The Jacobian contribution for the log transformation appears as

$$\log D' - \log D \quad (60)$$

in the Metropolis–Hastings ratio.

Changing D alters the OU relaxation matrix because

$$\mathbf{A} = \exp \left[-\frac{D\Delta t}{k_B T} \mathbf{K} \right], \quad (61)$$

and therefore changes the likelihood across all active dwells. The code recomputes the OU matrices for each dwell before evaluating the proposed likelihood.

16 Update 4: dwell-time hyperparameters

The function `sample_dwell_gamma()` updates two hyperparameters of the dwell-duration distribution: the Gamma shape α and mean μ . The active dwell durations enter the likelihood through

$$\Delta\tau_j \sim \text{Gamma} \left(\alpha, \frac{\mu}{\alpha} \right), \quad (62)$$

so that the Gamma scale is μ/α and the mean is exactly μ .

The current code proposes the mean with a log-normal random walk,

$$\log \mu' \sim \mathcal{N}(\log \mu, s_\gamma^2), \quad (63)$$

with $s_\gamma = 0.5$.

The shape parameter is treated differently in the updated implementation. Rather than using a log-normal random walk, the proposal is a discrete draw from the values

$$\alpha' \in \{1, 2, 3, 4, 5\}. \quad (64)$$

In code this appears as a random draw from `collect(dwell_gamma_shape_uniform_prior_min:dwell_gamma_shape_uniform_prior_max)`. The parameter file sets the lower and upper bounds to 1 and 5.

The source contains commented-out Gamma/Uniform prior expressions for α . In the active code, the explicit prior contribution for the shape parameter is commented out, while the Gamma prior on the mean remains active:

$$\mu \sim \text{Gamma}(1, \text{dwell_gamma_mean_prior_scale}). \quad (65)$$

This distinction matters: the discrete proposal range for α is an implementation constraint, but the active posterior code does not currently add a separate $\log p(\alpha)$ term. When modifying the method, this should be checked deliberately rather than assumed.

The acceptance calculation now also includes the annealing factor

$$\frac{1}{T_{\text{anneal}}} (\log \mathcal{P}' - \log \mathcal{P}) \quad (66)$$

plus the logarithmic proposal Jacobian terms associated with the positive-valued log-space proposal.

17 Update 5: change-point times

The function `sample_change_points_time!()` updates all candidate change-point times jointly.

17.1 Why use dwell durations?

Suppose the current dwell durations are

$$\Delta_j = \tau_{j+1} - \tau_j, \quad j = 1, \dots, J. \quad (67)$$

They satisfy

$$\sum_{j=1}^J \Delta_j = T. \quad (68)$$

The code normalizes the current durations by the total duration and uses a Dirichlet distribution to propose a new set of fractions. In conceptual form,

$$\mathbf{q}' \sim \text{Dirichlet}(\mathbf{1} + c\mathbf{q}), \quad (69)$$

where $c = 1000$ is the concentration parameter. The proposed durations are

$$\Delta'_j = Tq'_j. \quad (70)$$

The new change-point times are then reconstructed cumulatively.

17.2 Proposal validity rule

The code rejects proposals containing a dwell duration smaller than $2\Delta t$. This is a practical identifiability restriction: an inferred dwell cannot be shorter than roughly two sampling intervals in this implementation.

17.3 Asymmetric proposal correction

The Dirichlet proposal is state-dependent, so the acceptance ratio contains

$$\log q(\Theta \mid \Theta') - \log q(\Theta' \mid \Theta), \quad (71)$$

which the source labels as the backward minus forward transition probability.

This is necessary for the Markov chain to target the intended posterior rather than a distribution distorted by the proposal mechanism.

18 Update 6: active/inactive change points

This is the defining step of the nonparametric model. The function `sample_loads!()` considers each internal candidate change point and toggles it between active and inactive.

18.1 Prior probability of activation

The main program computes a Beta–Bernoulli-inspired activation probability

$$p_{\text{on}} = \frac{1}{1 + \frac{N_{\text{max}} - 3}{N_{\text{expected}} - 2}}, \quad (72)$$

using the chosen maximum and expected number of change points. The code stores

$$\log p_{\text{on}}, \quad \log(1 - p_{\text{on}}). \quad (73)$$

An activation proposal switches a candidate from zero to its own index; a deactivation proposal sets it back to zero.

18.2 Turning a change point on

When a previously inactive point is proposed active, the code splits one dwell into two. It evaluates:

1. the old single-dwell OU likelihood,
2. the old dwell-time prior,
3. the proposed first-dwell OU likelihood,
4. the proposed second-dwell OU likelihood,
5. the two proposed dwell-time priors,
6. geometric validity conditions for the inferred intermediate position.

The proposed intermediate position must lie within the coordinate extrema of the corresponding observed time segment. Otherwise the proposed likelihood is set to $-\infty$.

18.3 Turning a change point off

When an active change point is switched off, two adjacent dwells are merged into one. The merged dwell gets one OU likelihood and one dwell-time prior. The code also prevents the resulting direct jump from exceeding

$$\ell_{\text{max}} = 50 \text{ nm}. \quad (74)$$

The acceptance decision then compares the posterior of the merged model to the original two-dwell model.

18.4 Why this is step detection

This activation/deactivation mechanism is what turns the MCMC from a parameter estimator into a step-segmentation algorithm. The code is not merely estimating the positions of known steps; it is allowing the number and locations of the steps to change during inference.

19 Optional trajectory collapse rule

The nonparametric mode enables an explicit collapse operation intended to remove candidate steps whose inferred spatial separation is too small to be useful.

The updated parameter file defines

$$\ell_{\min} = 5 \text{ nm}, \quad (75)$$

$$\ell_{\max} = 50 \text{ nm}. \quad (76)$$

During the collapse stage, an active change point is considered for removal when the jump to the preceding active dwell is smaller than $\ell_{\min}$. When such a collapse occurs, the current change point is deactivated and the preceding latent dwell position is replaced by the midpoint of the two neighboring positions.

The updated main loop can trigger the collapse check at two stages of the annealing cycle: at the end of the initial burn-in and again halfway through the subsequent collapsed-burn-in interval. This is a change from the earlier version, which performed the collapse check at only one boundary.

The interpretation is important: this is an explicit structural intervention in the chain, not an ordinary reversible Metropolis proposal. It should therefore be documented as part of the analysis protocol and included in any reproducibility record.

The code also defines `annealing_constant_after_collapse`. In the supplied current `main.jl`, however, this parameter is not subsequently used in the annealing-temperature expression. The active temperature schedule continues to use `annealing_constant`. Thus, users should not assume that a separate post-collapse annealing constant is presently affecting the sampler.

20 Data validity and numerical safeguards

Several important numerical checks are built into the code.

20.1 Positive-definite stiffness

The likelihood returns $-\infty$ if

$$\det(\mathbf{K}) \leq 0 \quad (77)$$

or if the determinant is not finite.

20.2 Positive-definite OU covariance

The same strategy is used for Σ . If

$$\det(\Sigma) \leq 0 \quad (78)$$

or the determinant is non-finite, the log-likelihood becomes $-\infty$.

20.3 Position range restrictions

Proposed dwell positions must fall within the observed x - and y -ranges of their relevant segment.

20.4 Dwell-duration restrictions

Dirichlet proposals are invalidated if any proposed dwell is no longer than two frame intervals in the sense coded by

$$\Delta'_j \leq 2\Delta t. \quad (79)$$

20.5 Maximum jump restriction

Merged dwells are rejected if the implied jump is larger than 50 nm.

21 MCMC bookkeeping and acceptance diagnostics

The program tracks acceptance indicators for six classes of moves:

1. change-point times,
2. positions,
3. diffusion coefficient,
4. dwell-time hyperparameters,
5. stiffness,
6. active/inactive change-point indicators.

A moving window of length 100 is used. For the first 100 draws, the acceptance ratio is computed over the draws seen so far. Afterward, the arrays are shifted with `circshift!()` and the ratio is computed from the most recent 100 moves.

A practical interpretation is:

$$\text{acceptance ratio} \approx \frac{\text{number of accepted proposals in recent window}}{\text{number of proposals in recent window}}. \quad (80)$$

These diagnostics are useful for tuning proposal widths. Extremely low acceptance can mean proposals are too large; extremely high acceptance can mean proposals are too small and the chain may mix slowly. The supplied manual parameters should therefore be treated as experiment-specific rather than universal defaults.

22 Sampling schedule and saved posterior state

The code sets

$$N_{\text{saved}} = 200 \quad (81)$$

and

$$N_{\text{draws}} = N_{\text{saved}} \times N_{\text{save,burn}}. \quad (82)$$

With the current settings and collapse enabled,

$$N_{\text{burn}} = 11 \times 1000 = 11000, \quad (83)$$

$$N_{\text{collapsed burn}} = 12 \times 1000 - 11000 = 1000, \quad (84)$$

$$N_{\text{save,burn}} = 12000, \quad (85)$$

$$N_{\text{draws}} = 2,400,000. \quad (86)$$

Thus the program performs 2.4 million MCMC draws and saves one state every 12,000 draws, resulting in 200 saved states.

The posterior samples are stored in arrays for:

- active/inactive change-point states,
- change-point times,
- dwell positions,
- k_{xx} , k_{yy} , and k_{xy} ,
- diffusion coefficient,
- dwell-time Gamma mean and shape.

The log posterior is stored at every draw.

23 Output file structure

The function `save_mcmc_data()` writes an HDF5 file named

`mcmc_output_<file_id>.h5`

with datasets including:

Dataset	Meaning
<code>mcmc_loads</code>	Active/inactive candidate change-point representation for every saved sample.
<code>mcmc_change_points</code>	Saved change-point times.
<code>mcmc_dwell_gamma_shape</code>	Posterior samples of the dwell-time Gamma shape hyperparameter.
<code>mcmc_dwell_gamma_mean</code>	Posterior samples of the dwell-time Gamma mean.
<code>mcmc_position_x</code>	Posterior samples of latent dwell x -positions.
<code>mcmc_position_y</code>	Posterior samples of latent dwell y -positions.
<code>mcmc_k_xx</code>	Posterior samples of k_{xx} .
<code>mcmc_k_yy</code>	Posterior samples of k_{yy} .
<code>mcmc_k_xy</code>	Posterior samples of k_{xy} .
<code>mcmc_diffusion_coeff</code>	Posterior samples of D .
<code>mcmc_log_posterior</code>	Log-posterior trace at each MCMC draw.

24 Plotting and what the figures mean

When `plotting_on=true`, the code produces a three-panel figure.

24.1 Panel 1: trajectory and inferred dwell positions

The measured trajectory is plotted in nanometers. The inferred latent dwell positions are overlaid. If ground truth is available, the code also overlays the true positions.

This panel should answer:

Do the inferred dwell locations lie along the measured trajectory in physically plausible places, and do adjacent inferred dwells look like discrete steps rather than arbitrary noise fluctuations?

24.2 Panel 2: change-point locations in time

The active inferred change-point times are displayed. If ground truth is available, the true change-point times are also displayed.

The quality of step detection is therefore assessed by comparing posterior concentration of change-point times against the expected physical stepping events.

24.3 Panel 3: log-posterior trace

The log posterior is plotted versus MCMC draw. In a well-behaved run, after burn-in and annealing, one generally hopes to see the trace settle into a statistically stable region rather than continuing to drift systematically.

The code also prints the current annealing temperature, inferred positions, jump sizes, number of active change points, change-point times and indices, diffusion coefficient, dwell-time hyperparameters, stiffness parameters, all acceptance ratios, and the log posterior at the requested output frequency.

25 Posterior plotting workflow: `plot_mcmc_output.jl`

The additional plotting script is a downstream analysis program. It does not rerun the MCMC. Instead, it reads the original trajectory and the HDF5 posterior output written by `save_mcmc_data()`, reconstructs the active dwell quantities from every saved MCMC sample, and summarizes those quantities visually.

25.1 Packages used by the plotting script

The plotting script loads:

```
using CairoMakie, KernelDensity
using HDF5
using Distributions
using Statistics
```

The corresponding third-party packages are therefore `CairoMakie`, `KernelDensity`, `HDF5`, and `Distributions`. `Statistics` is part of Julia's standard library.

25.2 File locations

The plotting script defines

```
current_directory = "/home/mbgnjasb/codes/.../testing_2/"
```

This is an absolute path from the original analysis environment and must be changed for another computer. The script expects two HDF5 files in that directory:

1. the trajectory file `<file_id>.h5`, containing at least `x(m)` and `y(m)`;
2. the MCMC output file `mcmc_output_<file_id>.h5`.

The `file_id` and `data_range` are inherited from `input_parameters.jl`, so the plotting script should generally be run with the same parameter file used for the inference.

25.3 Reading the posterior HDF5 file

The script reads the datasets written by the inference code:

Dataset	Meaning
mcmc_loads	Active/inactive change-point state for each saved sample.
mcmc_change_points	Candidate change-point times.
mcmc_dwell_gamma_shape	Gamma dwell-time shape samples.
mcmc_dwell_gamma_mean	Gamma dwell-time mean samples.
mcmc_position_x, mcmc_position_y	Latent dwell positions.
mcmc_k_xx, mcmc_k_yy, mcmc_k_xy	Dwell-specific stiffness parameters.
mcmc_diffusion_coeff	Diffusion coefficient samples.
mcmc_log_posterior	Log posterior trace across MCMC draws.

Table 3: Posterior fields consumed by the plotting workflow.

25.4 Reconstructing active dwells

The plotting script repeats the same active/inactive bookkeeping used by the inference code. For each saved sample, the `loads` vector is copied from `mcmc_loads`, then the active change-point indices are identified.

For each active change point j , the script stores the corresponding change-point time. For every non-final change point it additionally constructs the dwell duration

$$\Delta\tau_j = \tau_{j+1} - \tau_j, \quad (87)$$

and records the corresponding latent position and stiffness parameters.

For internal transitions, the script computes the jump magnitude

$$\ell_j = \sqrt{(x_{j+1} - x_j)^2 + (y_{j+1} - y_j)^2}. \quad (88)$$

Because the script concatenates these quantities across all posterior samples, the resulting arrays represent the empirical posterior population of active dwells and jumps rather than one particular segmentation.

25.5 The Gamma fit shown in the dwell-time panel

The script uses

```
dist = fit(Gamma, Float64.(dwell))
```

to obtain a maximum-likelihood Gamma fit to the pooled posterior dwell durations. Thus, `dist.`

`alpha` and `dist.`

`theta` are empirical fitted parameters for the collected durations.

However, the curve actually plotted is constructed as

```
gamma_pdf = pdf.(Gamma.(dwell_fitted_gamma_shape .+ 0.4,
                        dwell_fitted_gamma_scale .- 0.002),
                dwell_grid)
```

and the annotation printed on the figure is also hard-coded. Therefore the red visual curve is *not exactly the maximum-likelihood Gamma fit returned by `fit(Gamma, dwells)`*. This should be treated as a presentation choice in the current script, not as a mathematically exact fit. If quantitative parameter reporting is desired, report the values in `dist`.

`alpha` and `dist`.

`theta` directly or change the plotted curve to use those parameters without modification.

25.6 Panel 1: posterior spatial density

The first panel estimates a two-dimensional kernel density from the pooled posterior dwell positions:

```
den = kde((position_x .* 1.0e9, position_y .* 1.0e9),
          npoints = (2048, 2048),
          bandwidth = (2.0, 2.0))
```

The positions are converted from meters to nanometers before KDE evaluation. The heatmap therefore shows where the inferred dwell positions accumulate across all saved posterior samples. The measured trajectory is superimposed as a white line.

This panel is best interpreted as a posterior occupancy map of inferred dwell locations, not as a single best-fit path. A bright region indicates that many posterior samples assign a dwell position there.

25.7 Panel 2: change-point histogram

The second panel is a histogram of all active change-point times collected over posterior samples. If the inference is stable, real transitions should tend to create persistent peaks in this histogram. A broad peak indicates uncertainty in transition timing; a nearly uniform background can indicate weak evidence for a particular change point.

When ground truth is available, the script adds vertical lines at the supplied ground-truth change-point times.

25.8 Panel 3: dwell-time distribution

The dwell-time panel shows a histogram of all inferred active dwell durations, normalized as a PDF, together with the current visual Gamma curve described above.

This is useful for checking whether the pooled inferred dwell durations have a unimodal, approximately Gamma-like structure. It also reveals unusually short or long dwell-time tails that may be hidden by a single posterior mean.

25.9 Panel 4: jump-size distribution

The jump-size panel displays a normalized histogram of posterior jump lengths in nanometers. This should be compared with the prior center of 8 nm and the hard 5–50 nm structural limits used in the updated inference workflow.

A strong concentration near the lower threshold can be a warning that the collapse rule or numerical bound is materially shaping the inferred step population.

25.10 Panels 5 and 6: dwell-time hyperparameters

The fifth panel shows the posterior distribution of the Gamma shape parameter α , while the sixth shows the posterior distribution of the Gamma mean parameter μ in seconds.

These are hyperparameters shared across active dwells. They should not be confused with individual dwell durations. For example, μ is the center of the prior distribution on dwell durations, whereas the $\Delta\tau_j$ values are individual inferred dwell times.

25.11 How to run the plotting script safely

A practical sequence is:

1. complete an inference run and verify that `mcmc_output_<file_id>.h5` exists;
2. make sure `input_parameters.jl` describes the same `file_id` and `data_range`;
3. edit `current_directory` in `plot_mcmc_output.jl`;
4. run `include("plot_mcmc_output.jl")` from Julia;
5. inspect the six-panel figure, especially the posterior change-point distribution, dwell-time distribution, and jump-size distribution;
6. use the posterior arrays in the HDF5 file for quantitative summaries rather than estimating numbers by eye from the figure.

25.12 Current plotting-script limitations

The supplied plotting script is a visualization script rather than a general analysis package. Several details are therefore intentionally fixed in the source: the KDE bandwidth and grid resolution, histogram bin counts, axis limits, figure size, font, and output filename. The script also assumes the title refers to “Dynein” even though the broader inference manual concerns kinesin and molecular motors generally. That title is presentation text and does not change the underlying inference model.

Finally, the plotting script reads `mcmc_log_posterior` but does not currently plot or otherwise use it. Likewise, it reads the posterior stiffness and diffusion arrays and uses stiffness arrays in the pooled dwell extraction, but it does not make dedicated stiffness or diffusion panels. Those quantities remain available in the HDF5 output for separate posterior analysis.

26 Ground-truth mode

The code can optionally load ground-truth quantities from the same HDF5 file when `ground_truth_available=`. These include:

- true x positions,
- true y positions,
- true change-point times,
- true dwell durations,
- true jump sizes.

The source also sets fixed reference values for ground-truth stiffness and diffusion in that branch. In the currently supplied parameter file, ground-truth mode is disabled.

Ground-truth mode is especially useful for simulated trajectories because it permits direct benchmarking of:

inferred step time $\leftrightarrow$ true step time,

inferred jump size $\leftrightarrow$ true jump size,

and

inferred D , $\mathbf{K} \leftrightarrow$ known generating values.

27 A concrete conceptual example

Suppose a motor takes the following idealized sequence along a microtubule:

$$\mathbf{r}_1 \rightarrow \mathbf{r}_2 \rightarrow \mathbf{r}_3 \rightarrow \mathbf{r}_4.$$

The measured trajectory, however, does not sit exactly at these locations. Instead, during dwell 1 the nanoparticle fluctuates around $\mathbf{r}_1$, then transitions to a noisy cloud around $\mathbf{r}_2$, and so forth.

The algorithm does not simply average each cloud. It asks whether a candidate segmentation and parameter set jointly explain the entire time series through the OU model.

For one dwell, the reasoning is approximately:

1. propose a latent center $\mathbf{r}_j$,
2. propose a stiffness matrix $\mathbf{K}_j$,
3. compute the corresponding relaxation matrix $\mathbf{A}_j$,
4. compute the covariance Σ_j ,
5. evaluate all within-dwell residuals,
6. add the dwell-time and jump-size priors,
7. compare this state with an alternative state,
8. accept or reject according to the Metropolis–Hastings rule.

At the segmentation level, the algorithm may also ask whether a pair of neighboring dwells is better represented as one dwell. That is the role of the active/inactive change-point sampler.

28 Practical interpretation of the inferred quantities

28.1 Step positions

The saved `mcmc_position_x` and `mcmc_position_y` arrays contain latent dwell positions. Posterior means, medians, credible intervals, or highest-posterior-density regions can be computed from these samples.

28.2 Step sizes

The code itself calculates Euclidean jump sizes as

$$\ell_j = \sqrt{(x_{j+1} - x_j)^2 + (y_{j+1} - y_j)^2}. \quad (89)$$

This is a geometrical step length in the measured 2D plane.

28.3 Step times and dwell times

The inferred dwell time is

$$\Delta\tau_j = \tau_{j+1} - \tau_j. \quad (90)$$

These are directly tied to the active change-point states, so uncertainty in whether a change point exists necessarily propagates into uncertainty in dwell times.

28.4 Stiffness

The diagonal elements describe directional restoring strength in the OU model. The off-diagonal term indicates coupling between the two coordinate directions. In an experimental setup where one axis aligns with a preferred mechanical direction, interpreting k_{xx} and k_{yy} individually requires remembering that the code uses the detector coordinate system unless the input has already been rotated.

28.5 Diffusion

The inferred D controls the stochastic mobility scale in the OU dynamics. It is not a detector-noise variance parameter. It is a dynamical parameter in the physical stochastic model.

29 Important implementation details and caveats

The following points are important when using the program scientifically.

29.1 The detector model is absent

The code starts from HDF5 Cartesian positions. Therefore, if quadrant-detector localization uncertainty is important, the present implementation does not explicitly separate:

$$\text{biophysical stochastic motion} \quad \text{from} \quad \text{measurement/localization noise.} \quad (91)$$

Those two effects can be statistically confounded unless the OU residual model and sampling cadence are sufficient to distinguish them.

29.2 Coordinate system matters

Because the code permits a full stiffness matrix, the inferred k_{xy} can depend on coordinate rotation. If the physical microtubule axis is known, an experimenter may want to rotate trajectories before analysis or interpret the inferred matrix in the detector frame carefully.

29.3 The prior scale is experiment-specific

The values 8 nm for the typical jump, 50 nm for the maximum allowed jump, 0.00075 N/m for typical stiffness, and the chosen dwell-time prior are not universal constants of kinesin. They are modeling choices in the supplied parameter file.

29.4 The first and last 100 samples are special

The code uses the first and last 100 observations to build endpoint position priors. This is useful when the trace begins and ends in relatively stable dwells, but it can become inappropriate if the trace begins or ends during a transition.

29.5 The MCMC state is repeatedly reinitialized after saving

After each saved sample block, the source calls `initialize_parameters!()` and rebuilds the OU matrices. Thus the saved samples are not generated by one uninterrupted chain that simply continues forever. The source intentionally restarts from an initialization after each save block.

This is a major point for downstream statistical analysis: the saved 200 states should not automatically be treated as 200 consecutive, equally spaced draws from a single stationary chain without checking how the restart strategy affects dependence and convergence.

29.6 Acceptance-ratio code contains asymmetric behavior between active and inactive states

For inactive-dwell stiffness and position updates, the likelihood contribution is intentionally set to zero in portions of the code because those candidate dwells are not currently active. Their values are therefore largely bookkeeping/placeholder values until activation. This is consistent with the nonparametric representation but means those inactive parameters should not be interpreted as posterior estimates of physical states.

29.7 There are legacy/commented likelihood implementations

`function_layer_2.jl` contains commented-out alternatives based on finite-correlation-length fractional Brownian motion and earlier Gaussian likelihood formulations. These are not executed by the supplied program. The active likelihood is the exact 2D OU implementation in `get_log_likelihood!()`.

30 How the complete algorithm can be summarized mathematically

Let the full latent state be

$$\Theta = \{\{\mathbf{r}_j\}, \{\tau_j\}, \{\mathbf{K}_j\}, D, \alpha, \mu, \{z_j\}\}, \quad (92)$$

where $z_j \in \{0, 1\}$ indicates whether candidate change point j is active.

For each active dwell, the observation model is approximately

$$\mathbf{y}_i - \mathbf{r}_j = \mathbf{A}_j(\mathbf{y}_{i-1} - \mathbf{r}_j) + \boldsymbol{\epsilon}_i, \quad (93)$$

with

$$\boldsymbol{\epsilon}_i \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_j). \quad (94)$$

The overall log posterior has the structure

$$\log \mathcal{P}(\Theta | y) = \sum_{j \in \text{active}} \log L_j + \sum_{j \in \text{active}} \log p(\Delta\tau_j | \alpha, \mu) \quad (95)$$

$$+ \sum_j \log p(\ell_j) + \sum_{j \in \text{active}} \log p(\mathbf{K}_j) \quad (96)$$

$$+ \log p(D) + \log p(\alpha) + \log p(\mu) + \sum_j \log p(z_j). \quad (97)$$

The MCMC then repeatedly modifies subsets of Θ and accepts or rejects the modifications.

31 Recommended workflow for a real experimental trajectory

A sensible analysis workflow around this code is:

1. Verify that the HDF5 `x(m)` and `y(m)` arrays are correctly calibrated and sampled at the intended Δt .
2. Plot the raw trajectory independently before running the sampler.

3. Check whether the trajectory begins and ends in genuine dwells if using the built-in first/last-100-point anchoring.
4. Choose physically defensible values for the jump-size, dwell-time, stiffness, and diffusion priors.
5. Run simulated trajectories with known ground truth to quantify bias and false step detection.
6. Examine the acceptance ratios for each proposal type.
7. Examine the log-posterior trace and the distribution of inferred numbers of active dwells.
8. Inspect posterior uncertainty of change-point times, not only the posterior mean trajectory.
9. Only then summarize step sizes, dwell times, and mechanical parameters across experimental traces.

32 A compact pseudocode version

Listing 1: Conceptual pseudocode matching the supplied implementation.

```

read x(t), y(t) from HDF5
center x and y so the first point is zero

initialize candidate dwell positions
initialize candidate change-point times
initialize active/inactive change points
initialize K_j, D, and dwell-time hyperparameters
precompute OU matrices for all candidate dwells

for draw = 1:total_draws
    compute annealing temperature

    # Continuous latent parameters
    propose and sample dwell positions
    propose and sample stiffness matrices
    propose and sample diffusion coefficient
    propose and sample dwell-time hyperparameters
    propose and sample change-point times

    # Model structure
    if nonparametric and within the designated burn-in phase
        propose turning candidate change points on/off
    end

    if collapse rule is enabled and collapse time is reached
        merge adjacent states with jumps below threshold
    end

    update acceptance diagnostics
    save log posterior
    optionally plot

    if draw is a save point
        save latent state to HDF5
        reinitialize the MCMC state

```

```

end
end

```

33 One-page conceptual map

For teaching purposes, the entire program can be remembered as five layers:

1. **Observation:** a measured (x_i, y_i) trajectory arrives from the tracking experiment.
2. **Segmentation:** the chain proposes where one dwell ends and the next begins.
3. **Latent geometry:** each dwell receives a latent position and therefore a jump to the next dwell.
4. **Dwell dynamics:** each dwell gets $\mathbf{K}_j$ and the global trajectory gets D ; these define the OU likelihood.
5. **Bayesian exploration:** MCMC samples all of the above jointly, balancing likelihood, priors, structural constraints, and proposal probabilities.

The essential computational feedback loop is therefore

change points $\rightarrow$ dwell positions $\rightarrow$ stiffness and diffusion $\rightarrow$ OU likelihood $\rightarrow$ posterior $\rightarrow$ new change points

That loop is the core of the supplied implementation.

34 Source-code map for readers learning the program

The following map is intended as a quick reference when reading the Julia source.

Function / block	Read it as
<code>get_data()</code>	Load measured trajectory and shift the coordinate origin.
<code>initialize_parameters!()</code>	Construct the initial latent stepping model.
<code>get_active_inactive_loads!()</code>	Keep track of which candidate change points are active.
<code>prepare_OU!()</code>	Convert D and $\mathbf{K}$ into OU dynamics and covariance matrices.
<code>get_log_likelihood!()</code>	Score the observed trajectory under one dwell model.
<code>sample_positions!()</code>	Move latent dwell locations.
<code>sample_stiffness!()</code>	Move k_{xx}, k_{yy}, k_{xy} .
<code>sample_diffusion_coeff!()</code>	Move global diffusion coefficient D .
<code>sample_dwell_gamma!()</code>	Move the dwell-time prior parameters.
<code>sample_change_points_time!()</code>	Move the continuous change-point times.
<code>sample_loads!()</code>	Turn candidate change points on or off.
<code>plot_stuff()</code>	Visualize trajectory, inferred states, change points, and posterior trace.
<code>save_mcmc_data()</code>	Save posterior samples and diagnostics to HDF5.
<code>sampler()</code>	Orchestrate the entire MCMC sequence.

35 Final perspective

The most important conceptual point is that the code is doing *joint Bayesian step inference*, not merely thresholding a position trace. A candidate step must survive several simultaneous tests:

Does the proposed dwell position fit the observed positions?

Does the OU fluctuation model fit the time-correlated motion within the dwell?

Is the dwell duration plausible?

Is the jump size plausible?

Is the stiffness matrix physically valid?

Does the complete segmented model have higher posterior probability?

Because all of these ingredients are sampled together, the resulting uncertainty in step position and dwell time is, in principle, propagated through the mechanical model rather than being treated as a separate deterministic segmentation error.

At the same time, the implementation should be regarded as an experiment-specific inference engine. In particular, the absence of an explicit quadrant-detector observation model, the hand-selected priors and thresholds, the trajectory-collapse operation, and the reinitialization after each saved block are all modeling/design decisions that matter when interpreting the resulting posterior quantitatively.

A Current parameter set in the updated code

For convenient reference, the active configuration is summarized below. Values are taken directly from the updated source files.

Parameter	Value	Meaning
random_number_generator_seed	123	Reproducible RNG seed.
file_id	processed_trajectory_ invivo	Input HDF5 identifier.
data_range	1:920000	Current selected observations.
ground_truth_available	false	Ground-truth branch disabled.
T	300 K	Temperature.
dt	5×10^{-7} s	Sampling interval.
plotting_on	false	Live inference plotting disabled.
n_samples_to_save	200	Posterior states saved.
minimum_allowed_jump_size	5 nm	Lower jump/collapse threshold.
maximum_allowed_jump_size	50 nm	Upper direct-jump limit in merge checks.

modeling_choice	nonparametric	Variable active dwell count.
expected_n_dwells	10	Prior/target expected dwell count.
max_n_dwells	135 for current data	Data-duration-dependent candidate maximum.
max_n_change_points	136 for current data	One more than maximum dwells.
typical_dwell_gamma_shape	2	Initial Gamma shape.
typical_dwell_time	0.0175 s	Initial/typical dwell mean.
dwell_gamma_shape_uniform_prior_min		Lower allowed shape proposal.
dwell_gamma_shape_uniform_prior_max		Upper allowed shape proposal.
dwell_gamma_mean_prior_shape	1	Mean-hyperprior shape.
dwell_gamma_mean_prior_scale	0.0175 s	Mean-hyperprior scale.
typical_jump_size	8 nm	Jump-size prior center.
std_jump_size_prior	8 nm	Jump-size prior width.
typical_k_value	7.5×10^{-5} N/m	Typical stiffness.
k_gamma_prior_shape	1	Stiffness prior shape.
typical_diffusion_coeff	10^{-13} m ² /s	Typical diffusion coefficient.
std_diffusion_coeff_prior	10^{-11} m ² /s	Diffusion prior standard deviation.
conc_parameter_change_points_proposal	500	Dirichlet proposal concentration.
step_size_position_sampler	0.5 nm	Position proposal width.
step_size_diffusion_coeff_sampler	0.01	Log-diffusion proposal width.
step_size_dwell_gamma	0.5	Log-mean proposal width.
stiffness_gamma_proposal_alpha	500	Diagonal-stiffness proposal concentration.
stiffness_xy_proposal_alpha	500	Cross-correlation proposal concentration.
starting_temperature	20000	Initial annealing temperature.
annealing_constant	1000	Annealing decay constant.
burn_in_period	11000 draws	Initial burn-in in nonparametric mode.
collapsed_burn_in_period	1000 draws	Remaining interval in the current two-stage cycle.
save_burn_in_period	12000 draws	Interval between saved states.
save_frequency	12000 draws	Posterior-save interval.
total_draws	2,400,000	Total MCMC draws.
window_size	100	Acceptance-ratio moving window.

For the current trajectory, the total observation duration is

$$T_{\text{record}} = (920000 - 1) \times 5 \times 10^{-7} \approx 0.4599995 \text{ s.} \quad (98)$$

Because this candidate maximum is computed from `typical_dwell_time`, choosing a realistic approximate average dwell time is essential before running the nonparametric sampler.

The data-dependent candidate maximum is therefore

$$J_{\text{max}} = 5 \left\lceil \frac{0.4599995}{0.0175} \right\rceil = 135. \quad (99)$$

B Suggested downstream posterior analysis

The supplied code stops at writing posterior samples. A separate analysis script can then compute, for each posterior sample, the number of active dwells, jump sizes, dwell durations, and derived mechanical quantities. For example,

$$\ell_j^{(s)} = \sqrt{\left(x_{j+1}^{(s)} - x_j^{(s)}\right)^2 + \left(y_{j+1}^{(s)} - y_j^{(s)}\right)^2}, \quad (100)$$

and

$$\Delta\tau_j^{(s)} = \tau_{j+1}^{(s)} - \tau_j^{(s)}. \quad (101)$$

Posterior credible intervals for these derived quantities should be computed from the collection of posterior samples rather than from the single displayed MCMC trajectory.

End of manual.

References

- [1] Julia Programming Language, “Downloads”, official Julia downloads page. See <https://julialang.org/downloads/>. Accessed August 25, 2026.
- [2] Julia Programming Language, “Platform Specific Instructions for Official Binaries”, official Julia documentation. See <https://www.julialang.org/downloads/platform/>. Accessed August 25, 2026.
- [3] Julia Pkg documentation, “Managing Packages”, official package-management documentation. See <https://pkgdocs.julialang.org/dev/managing-packages/>. Accessed August 25, 2026.
- [4] Julia Pkg documentation, “Getting Started”, official package-management documentation. See <https://pkgdocs.julialang.org/v1/getting-started/>. Accessed August 25, 2026.